PacMan SDN Project
Live Demo Guide

Every command in order, what it does, what is inside each script, which files are used, where the change shows up, what to say, and the cross-questions to expect

Name	Roll no.
Gagan Rathod	23bcs106
Atulesh Sahoo	23bcs022
Ganesh Daware	23bcs038
Bhagya Vishwakarma	23bcs029
Dudlewar Adinath	23bcs046
Harshit	23bcs037

Group SDN_28 · IIIT Dharwad · Software Defined Networking

How to read each step

T2 · mininet> dark box	The exact command. Copy the whole box and paste it into the terminal on the purple label. `mininet>` boxes go at the Mininet prompt; `test.py ❯` boxes are typed into our demo console in T4.
DOES	What the command does, in plain words.
INSIDE	What happens inside the script or program that runs.
FILES	Which project files are run, read or written by this action.
LOOK	Where the change appears (T1, T2, T3, T4).
SAY	Easy words to say while showing it.
CROSS-Q	Short questions the professor may ask, with answers.

Folders

Folder / file	What it is
`~/Desktop/7sem/SDN/PROJECT`	our project. **All four terminals work here.**
`~/Desktop/7sem/SDN/pox`	POX controller (run inside a Podman container by `scripts/run_pox.sh`)
`~/Desktop/7sem/SDN/pacman/build/pacman`	PacMan program (started by `scripts/run_pacman.sh`)
`PROJECT/results/`	live outputs: `blocklist.txt`, `block_events.csv`, `flow_stats.csv`, `pacman.log`
`PROJECT/test.py`	our demo console for T4: a menu of tests; each one explains, runs and says where to look

NOTE All screenshots are from a real run of exactly these commands (10 Nov 2026) in kitty. Times, counters and packet numbers will differ in your run. The whole demo takes about 10 minutes, including the 2-minute block expiry. sudo does not ask for a password.

Part A · Start the system

Step 0 · Open the 4-terminal grid

Open kitty. Press `Ctrl+Shift+Enter` three times (4 terminals), then `Ctrl+Shift+L` until they form a 2×2 grid. Click a terminal to focus it.

T1 (top left) POX controller	**T2** (top right) Mininet network
T3 (bottom left) PacMan dashboard	**T4** (bottom right) test.py console

T1 · T2 · T3 · T4 (in each)

```
cd ~/Desktop/7sem/SDN/PROJECT
```

SAY "We use four terminals: T1 is the SDN controller, T2 is the network, T3 is our monitor PacMan, and T4 is where we run tests and attacks."

Step 1 · Clean start, open the test console

T4

```
sudo mn -c 2>/dev/null
podman rm -f pacman-pox
rm -f results/blocklist.txt results/block_events.csv
python3 test.py
```

DOES `mn -c` removes any leftover Mininet switches and links; `podman rm` removes an old controller container; `rm` deletes old block requests so nobody is blocked at start. `python3 test.py` opens our demo console with a menu of tests (0–14) and `live`.

INSIDE test.py keeps a list of tests. For each one it prints *What it does* and *Files used*, types each command after a green `T4 ❯` prompt and runs it in bash (so you see the real output), then prints *Where to look*. Enter shows the menu again; `clear` clears the screen; `q` quits.

FILES `test.py`

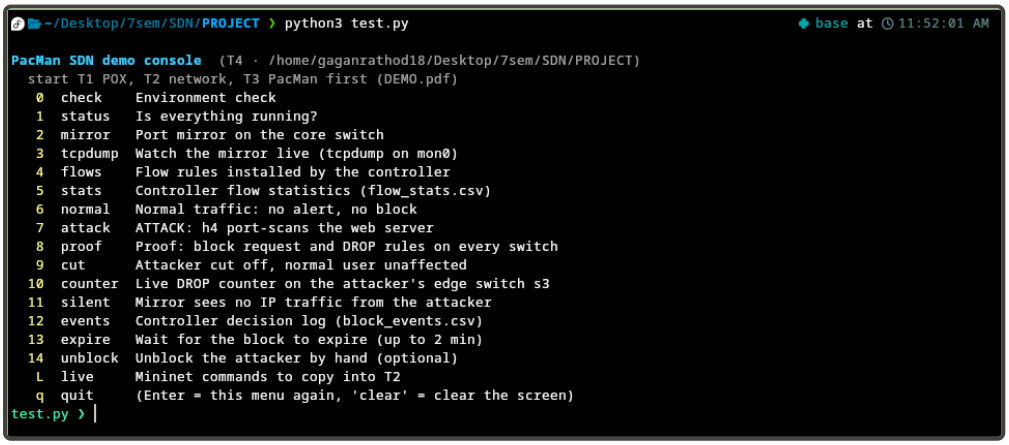


Figure 1. test.py menu in T4.

Step 2 · Environment check

T4 · test.py ❯

```
0
```

DOES Checks that every tool the project needs is installed.

INSIDE `scripts/check_env.sh` uses `command -v` for mn, ovs-vsctl, ovs-ofctl, podman, tcpdump, python3; tests passwordless sudo; runs `sudo python3 -c "import mininet"`; checks the openvswitch service with `systemctl is-active`, the `python:2.7` Podman image, `../pox/pox.py`, the PacMan binary, and that port 6633 is free (`ss -ltn`).

FILES `scripts/check_env.sh` (looks for `../pox/pox.py`, `../pacman/build/pacman`)

LOOK T4: every line **[OK]**, "All required components found."

SAY "First we check the setup: Mininet, Open vSwitch, the controller, and our monitor are all installed."

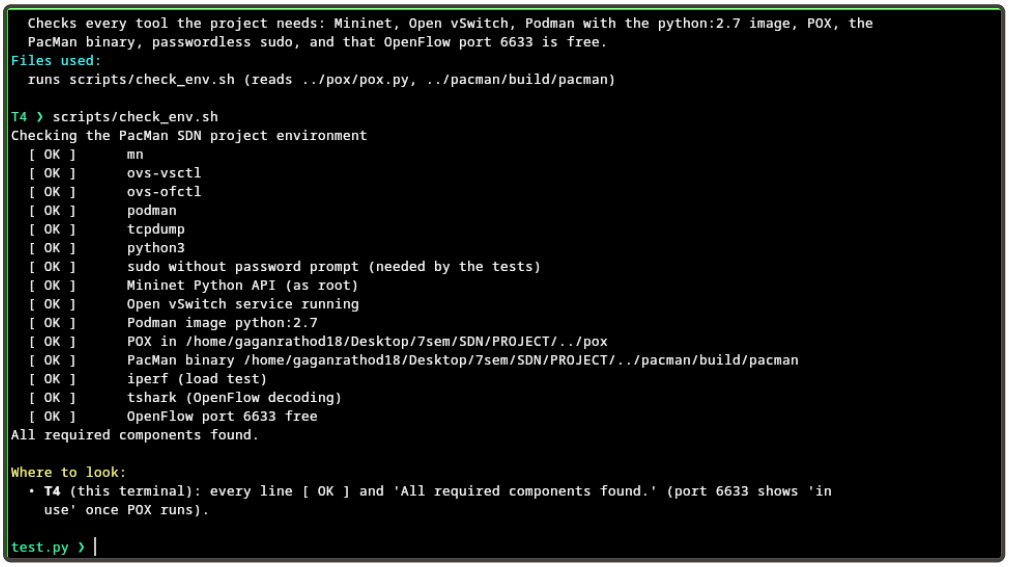


Figure 2. Environment check.

Step 3 · Start the SDN controller (control plane)

T1

```
scripts/run_pox.sh
```

DOES Starts the POX controller with two parts: `forwarding.l2_learning` (normal switching) and our `pacman_ips` (flow statistics + blocking). A block lasts 120 s.

INSIDE POX needs Python 2.7, so the script runs it in a Podman container:

```
podman run --rm --network host -v ../pox:/pox -v controller:/project/controller \
       -v results:/project/results python:2.7 \
  python ./pox.py forwarding.l2_learning pacman_ips --interval=5 --block_seconds=120 \
         --protect=10.0.0.1 --blocklist=/project/results/blocklist.txt ...
```

`--network host` lets the switches reach it at 127.0.0.1:6633; the `results` folder is shared, so the controller can read the blocklist that PacMan writes. Settings can change: `BLOCK_SECONDS=60 scripts/run_pox.sh`.

FILES `scripts/run_pox.sh` → `../pox/pox.py` + `controller/pacman_ips.py`; reads `results/blocklist.txt`; writes `results/flow_stats.csv`, `results/block_events.csv`

LOOK T1: "Flow monitor ready: polling every 5.0s", "Blocker ready ... block for 120.0s, protected: 10.0.0.1", "POX 0.2.0 (carp) is up."

SAY "This is the brain of the network. It decides how every switch forwards traffic, and it will also block attackers."

Q: Why Podman / Python 2.7?
POX 0.2.0 only runs on Python 2.7, so we run it in a small container instead of changing the laptop's Python.

Q: What is "protected 10.0.0.1"?
The web server is never blocked, even if someone pretends to be it.

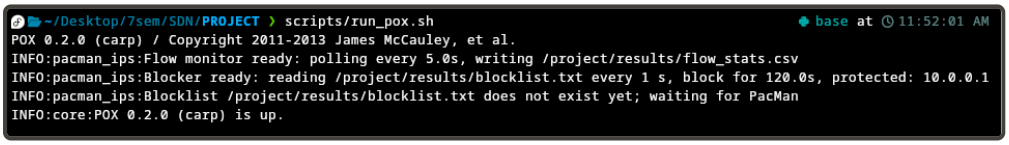


Figure 3. POX is up in T1.

Step 4 · Start the network (data plane)

T2

```
sudo python3 topology/campus_topo.py
```

DOES Builds the campus network: core switch s1, server switch s2 (h1 web 10.0.0.1, h2 files 10.0.0.2), user switch s3 (h3 user 10.0.0.3, h4 attacker 10.0.0.4); connects all switches to POX; makes s1 copy every packet to the port `mon0`.

INSIDE `CampusTopo.build()` calls `addSwitch`, `addHost(ip, mac)`, `addLink`; `build_net()` adds a `RemoteController` at 127.0.0.1:6633 and turns IPv6 off so captures stay clean; `add_mirror()` runs:

```
ovs-vsctl add-port s1 mon0 -- set interface mon0 type=internal
ovs-vsctl ... create mirror name=pacman select-all=true output-port=@p -- set bridge s1 mirrors=@m
ovs-ofctl mod-port s1 mon0 no-flood
```

then opens the Mininet CLI.

FILES `topology/campus_topo.py`

LOOK T2: "Mirror: all traffic on s1 is copied to mon0", host list, `mininet>`. T1: "Switch 00-00-00-00-00-0N connected (3 switch(es) monitored)".

SAY "Mininet creates a small campus network on this laptop: a core switch, a server zone and a user zone. Every packet through the core is copied to our monitor."

Q: Why mirror only the core?

All traffic between the zones passes through it. Traffic inside one zone (h1 ↔ h2) is not seen; that's a known limit.

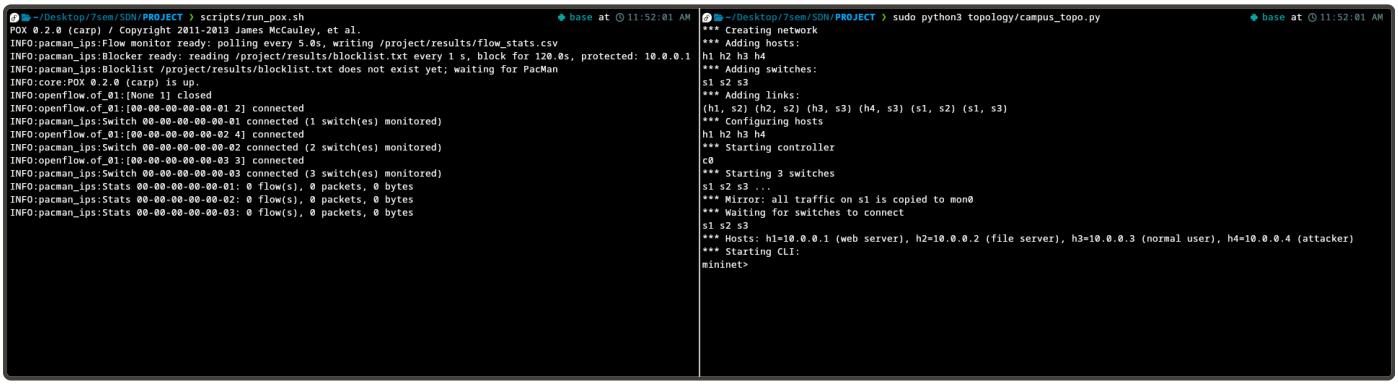


Figure 4. T1: switches connect · T2: network built.

Part B · Live in Mininet (T2)

Step 5 · Look at the network

T2 · mininet>

```
nodes
```

T2 · mininet>

```
links
```

T2 · mininet>

```
dump
```

DOES `nodes` lists the controller c0, hosts h1–h4 and switches s1–s3; `links` lists every cable and whether both ends are up (OK OK); `dump` shows each node's IP and process id, and c0 as a remote controller at 127.0.0.1:6633.

SAY "These are our 4 hosts and 3 switches. Every link is up, and the controller is the remote POX."

```
mininet> nodes
available nodes are:
c0 h1 h2 h3 h4 s1 s2 s3
mininet> links
h1-eth0<->s2-eth2 (OK OK)
h2-eth0<->s2-eth3 (OK OK)
h3-eth0<->s3-eth2 (OK OK)
h4-eth0<->s3-eth3 (OK OK)
s1-eth1<->s2-eth1 (OK OK)
s1-eth2<->s3-eth1 (OK OK)
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=61693>
<Host h2: h2-eth0:10.0.0.2 pid=61695>
<Host h3: h3-eth0:10.0.0.3 pid=61697>
<Host h4: h4-eth0:10.0.0.4 pid=61699>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=61704>
<OVSSwitch s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None,s2-eth3:None pid=61707>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None,s3-eth3:None pid=61710>
<RemoteController c0: 127.0.0.1:6633 pid=61785>
mininet>
```

Figure 5. nodes, links, dump.

Step 6 · Connectivity, and each host is its own machine

T2 · mininet>

```
pingall
```

T2 · mininet>

```
h4 ip -br addr
```

T2 · mininet>

```
h4 ip route
```

DOES `pingall`: every host pings every other host. `h4 …` runs the command **inside h4**: h4 only sees its own interface `h4-eth0` with 10.0.0.4.

INSIDE Each Mininet host is a Linux **network namespace** (its own interfaces, IP and routes) connected to a switch by a virtual cable (veth pair). The first ping of each pair goes to POX (PACKET_IN), and l2_learning installs a flow rule.

LOOK T2: "Results: 0% dropped (12/12 received)". T1: stats lines start showing flows.

SAY "All hosts can reach each other. Each host is like a separate small computer with its own network card."

Q: Who installs the forwarding rules?
POX's l2_learning: it learns which port each MAC address is on and installs a rule (idle 10 s, hard 30 s).

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> h4 ip -br addr
lo               UNKNOWN        127.0.0.1/8
h4-eth0@if58     UP             10.0.0.4/24
mininet> h4 ip route
10.0.0.0/24 dev h4-eth0 proto kernel scope link src 10.0.0.4
mininet>
```

Figure 6. pingall and h4's own interface.

Step 7 · Web servers, then start PacMan (application plane)

T2 · mininet>

```
h1 python3 -m http.server 80 --directory /tmp &
```

T2 · mininet>

```
h2 python3 -m http.server 8080 --directory /tmp &
```

T2 · mininet>

```
h3 curl -s -o /dev/null -w "h3 -> h1: HTTP %{http_code}" http://10.0.0.1/ ; echo
```

T2 · mininet>

```
h3 curl -s -o /dev/null -w "h3 -> h2: HTTP %{http_code}" http://10.0.0.2:8080/ ; echo
```

T3

```
scripts/run_pacman.sh
```

DOES Starts a web server on h1 (port 80) and h2 (port 8080) and checks them from h3 (HTTP 200). Then starts PacMan's live dashboard on the mirror port.

INSIDE `run_pacman.sh` checks the PacMan binary exists and `mon0` exists, creates `results/pacman.log` and `results/blocklist.txt` as you (so the controller can read them), then runs `sudo pacman live config/pacman-sdn.conf --iface mon0`. PacMan captures with libpcap, parses each packet, tracks flows and runs the detector; `firewall_mode=sdn` means HIGH alerts become block requests.

FILES `scripts/run_pacman.sh → ../pacman/build/pacman` ; reads `config/pacman-sdn.conf` ; writes `results/pacman.log` , `results/blocklist.txt` , `results/pacman-reports/`

LOOK T2: HTTP 200 twice. T3: dashboard with packet counters, flows, Alerts "HIGH 0". Keys: ←/→ tabs, ↑/↓ scroll, q quit.

SAY "Now our monitor PacMan is watching the copy of all core traffic. Right now everything is clean: no alerts."

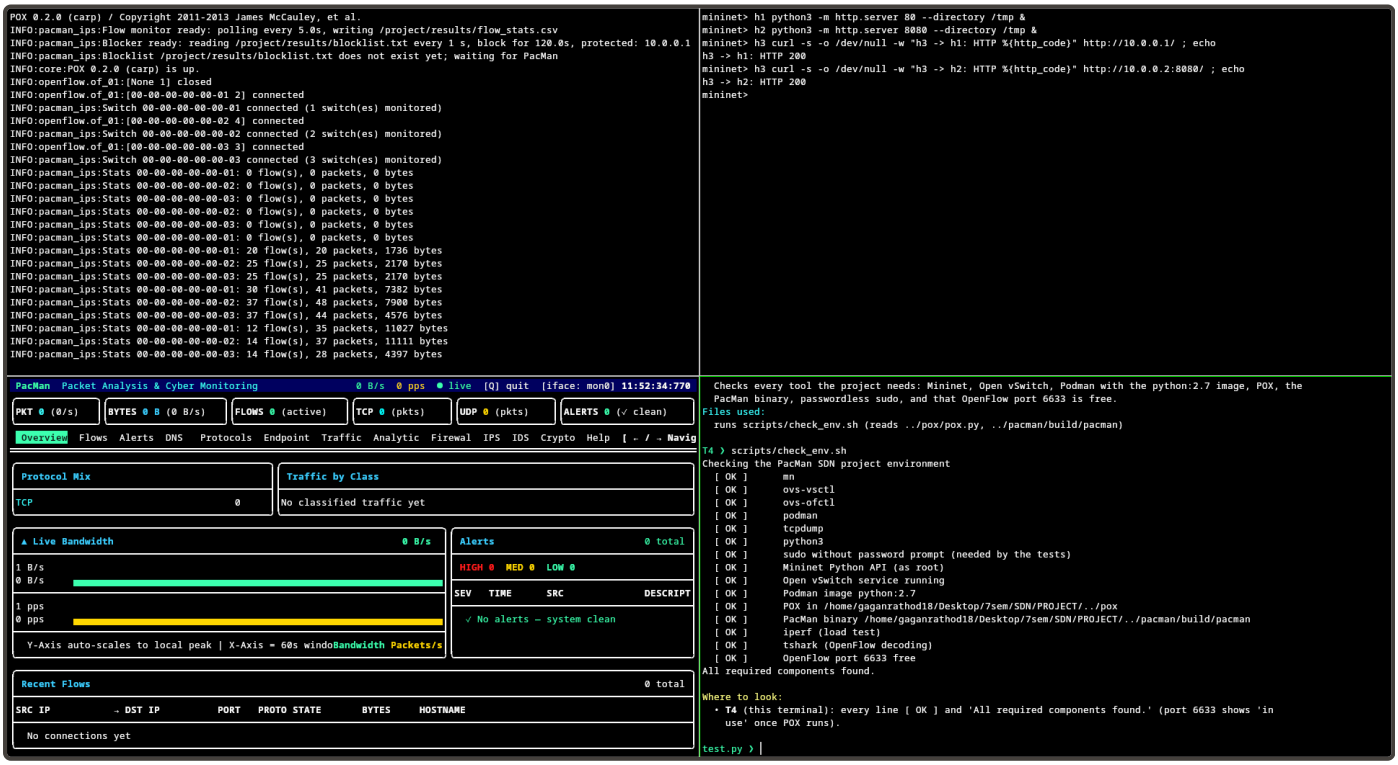


Figure 7. T2: web servers answer · T3: PacMan live on mon0.

Part C · Monitoring tests (T4, test.py)

Step 8 · Is everything running?

T4 · test.py ❯

```
1
```

DOES Runs `podman ps` (controller container), `ovs-vsctl list controller` (are the switches connected?), `pgrep` for the Mininet hosts and PacMan.

FILES none: asks Podman, Open vSwitch and the process list

LOOK T4: `pacman-pox Up`, three `is_connected : true`, h1–h4 shells, one PacMan process.

SAY "The controller is running, all three switches are connected to it, and PacMan is running."

```
── [1] Is everything running? ──
What it does:
  Shows the POX container, whether all 3 switches are connected to the controller, the Mininet host shells and
  the PacMan process.
Files used:
  asks Podman (container pacman-pox from scripts/run_pox.sh), Open vSwitch, and the process list

T4 ❯ podman ps --format '{{.Names}}  {{.Status}}'
pacman-pox  Up 29 seconds
T4 ❯ sudo ovs-vsctl --columns=_uuid,target,is_connected list controller | grep -E 'target|is_connected'
target              : "tcp:127.0.0.1:6633"
is_connected        : true
target              : "tcp:127.0.0.1:6633"
is_connected        : true
target              : "tcp:127.0.0.1:6633"
is_connected        : true
T4 ❯ pgrep -af 'mininet:h[1-4]$' | sed 's/^/  /'
  61693 bash --norc --noediting -is mininet:h1
  61695 bash --norc --noediting -is mininet:h2
  61697 bash --norc --noediting -is mininet:h3
  61699 bash --norc --noediting -is mininet:h4
T4 ❯ pgrep -ax pacman | sed 's/^/  /'
  62638 /home/gaganrathod18/Desktop/7sem/SDN/pacman/build/pacman live config/pacman-sdn.conf --iface mon0

Where to look:
  • T4 (this terminal): pacman-pox Up, three 'is_connected : true', h1-h4 shells, one PacMan process.

test.py ❯
```

Figure 8. Status check.

Step 9 · The port mirror

T4 · test.py ❯

```
2
```

DOES `sudo ovs-vsctl list mirror` shows the mirror; `sudo ovs-ofctl dump-ports-desc s1 | grep -A1 mon0` shows mon0's settings.

FILES mirror created by `topology/campus_topo.py` (add_mirror)

LOOK T4: `select_all : true`, statistics counting up; `mon0 … NO_FLOOD`.

SAY "The core switch copies all of its traffic to mon0. NO_FLOOD stops flooded packets being counted twice."

Q: Does the mirror slow the network?
No. The switch makes the copy; PacMan is not in the packet path, so if PacMan is slow the network is not.

Step 10 · Watch the mirror live with tcpdump

T4 · test.py ❯

```
3
```

DOES Starts `tcpdump` on mon0 for 8 ICMP packets while h3 pings h1 4 times (via `scripts/on_host.sh`).

INSIDE `on_host.sh h3 …` finds h3's shell process (`pgrep "mininet:h3"`) and runs the command inside h3's namespace with `sudo mnexec -a <pid>`. So T4 can act as any host.

FILES `scripts/on_host.sh` ; tcpdump reads `mon0`

LOOK T4: the **first** request and reply appear **twice**, later ones once. T3: PacMan's counters and "Recent Flows" show 10.0.0.3 ↔ 10.0.0.1.

SAY "Here we see the copied packets. The first packet appears twice because the switch had no rule yet: it sent the packet to the controller, and the controller sent it back out. After that a rule exists, so the packets go straight through."

Q: What are PACKET_IN and PACKET_OUT?

PACKET_IN: the switch asks the controller about an unknown packet. PACKET_OUT: the controller tells the switch to send that packet.

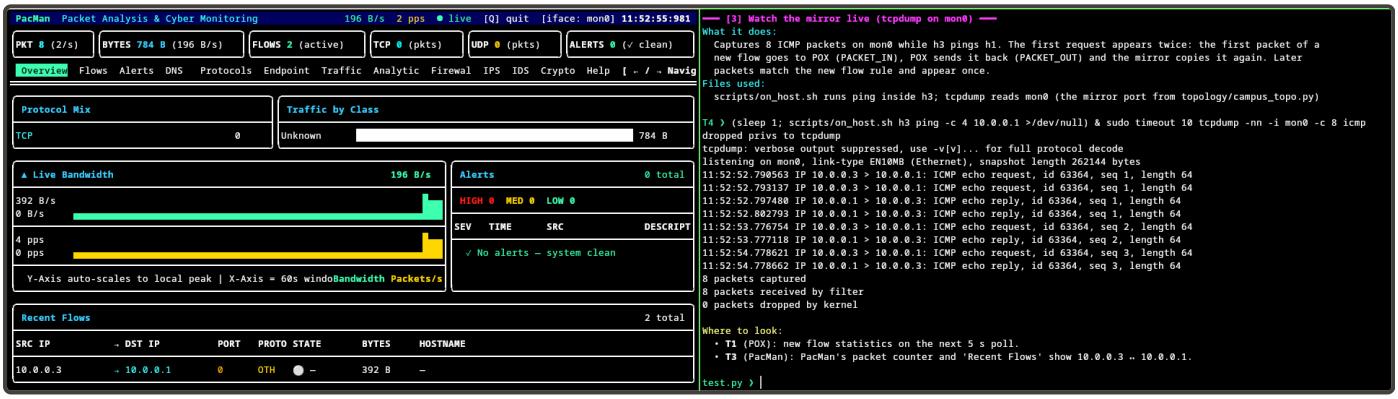


Figure 9. T3: PacMan sees the ping · T4: tcpdump on mon0 (first packet twice).

Step 11 · Flow rules made by the controller

T4 · test.py ❯

```
4
```

DOES h3 pings h1, then `sudo ovs-ofctl --no-stats dump-flows s1` prints s1's flow table.

FILES rules installed by `../pox/pox/forwarding/l2_learning.py`

LOOK T4: 2 rules (h3→h1, h1→h3) with `idle_timeout=10, hard_timeout=30, priority=65535` and `actions=output:…`.

SAY "The controller installed these rules. Each rule says: packets matching this go out of that port. They disappear 10 s after the traffic stops."

Q: Why priority 65535?

These are exact-match rules (all fields set). In OpenFlow 1.0 an exact match is always top priority, and OVS shows it as 65535. That's why our blocker deletes them before adding the drop rule.

```
T4 ❯ scripts/on_host.sh h3 ping -c 3 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.078 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.138 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.109 ms

--- 10.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2040ms
rtt min/avg/max/mdev = 0.078/0.108/0.138/0.024 ms
T4 ❯ sudo ovs-ofctl --no-stats dump-flows s1
 idle_timeout=10, hard_timeout=30, priority=65535,icmp,in_port="s1-eth2",vlan_tci=0x0000,dl_src=00:00:00:00:00:03,dl_ds
t=00:00:00:00:00:01,nw_src=10.0.0.3,nw_dst=10.0.0.1,nw_tos=0,icmp_type=8,icmp_code=0 actions=output:"s1-eth1"
 idle_timeout=10, hard_timeout=30, priority=65535,icmp,in_port="s1-eth1",vlan_tci=0x0000,dl_src=00:00:00:00:00:01,dl_ds
t=00:00:00:00:00:03,nw_src=10.0.0.1,nw_dst=10.0.0.3,nw_tos=0,icmp_type=0,icmp_code=0 actions=output:"s1-eth2"
 idle_timeout=10, hard_timeout=30, priority=65535,arp,in_port="s1-eth2",vlan_tci=0x0000,dl_src=00:00:00:00:00:03,dl_dst
=00:00:00:00:00:01,arp_spa=10.0.0.3,arp_tpa=10.0.0.1,arp_op=1 actions=output:"s1-eth1"
 idle_timeout=10, hard_timeout=30, priority=65535,arp,in_port="s1-eth1",vlan_tci=0x0000,dl_src=00:00:00:00:00:01,dl_dst
=00:00:00:00:00:03,arp_spa=10.0.0.1,arp_tpa=10.0.0.3,arp_op=2 actions=output:"s1-eth2"
 idle_timeout=10, hard_timeout=30, priority=65535,arp,in_port="s1-eth1",vlan_tci=0x0000,dl_src=00:00:00:00:00:01,dl_dst
=00:00:00:00:00:03,arp_spa=10.0.0.1,arp_tpa=10.0.0.3,arp_op=1 actions=output:"s1-eth2"
 idle_timeout=10, hard_timeout=30, priority=65535,arp,in_port="s1-eth2",vlan_tci=0x0000,dl_src=00:00:00:00:00:03,dl_dst
=00:00:00:00:00:01,arp_spa=10.0.0.3,arp_tpa=10.0.0.1,arp_op=2 actions=output:"s1-eth1"

Where to look:
  • T4 (this terminal): two rules (h3→h1 and h1→h3) with actions=output:… They disappear 10 s after
    the ping stops.

test.py ❯
```

Figure 10. s1's flow table.

Step 12 · Bandwidth test, and controller flow statistics

T2 · mininet>

```
iperf h3 h1
```

T4 · test.py ❯

```
5
```

DOES `iperf h3 h1` measures TCP bandwidth from h3 to h1 through the core. Test 5 shows the last rows of `results/flow_stats.csv` (time, switch, type, source, destination, packets, bytes).

INSIDE Every 5 s our `FlowMonitor` (in pacman_ips.py) sends an OpenFlow **flow stats request** to every switch and writes one CSV row per rule from the reply.

FILES `controller/pacman_ips.py` (FlowMonitor) → `results/flow_stats.csv`

LOOK T2: about 22 Gbit/s. T3: a **MEDIUM "Large transfer"** alert for 10.0.0.3, and **no block**. T4: the iperf flow with billions of bytes. T1: stats lines every 5 s.

SAY "A big download raises only a MEDIUM alert. It is shown to the admin but not blocked, because only HIGH alerts block. The controller also counts every flow on every switch, which is the SDN side of the monitoring."

Q: Why not block big transfers?
Big downloads are normal. Blocking is only for clear attacks (scan, flood), to avoid false alarms.

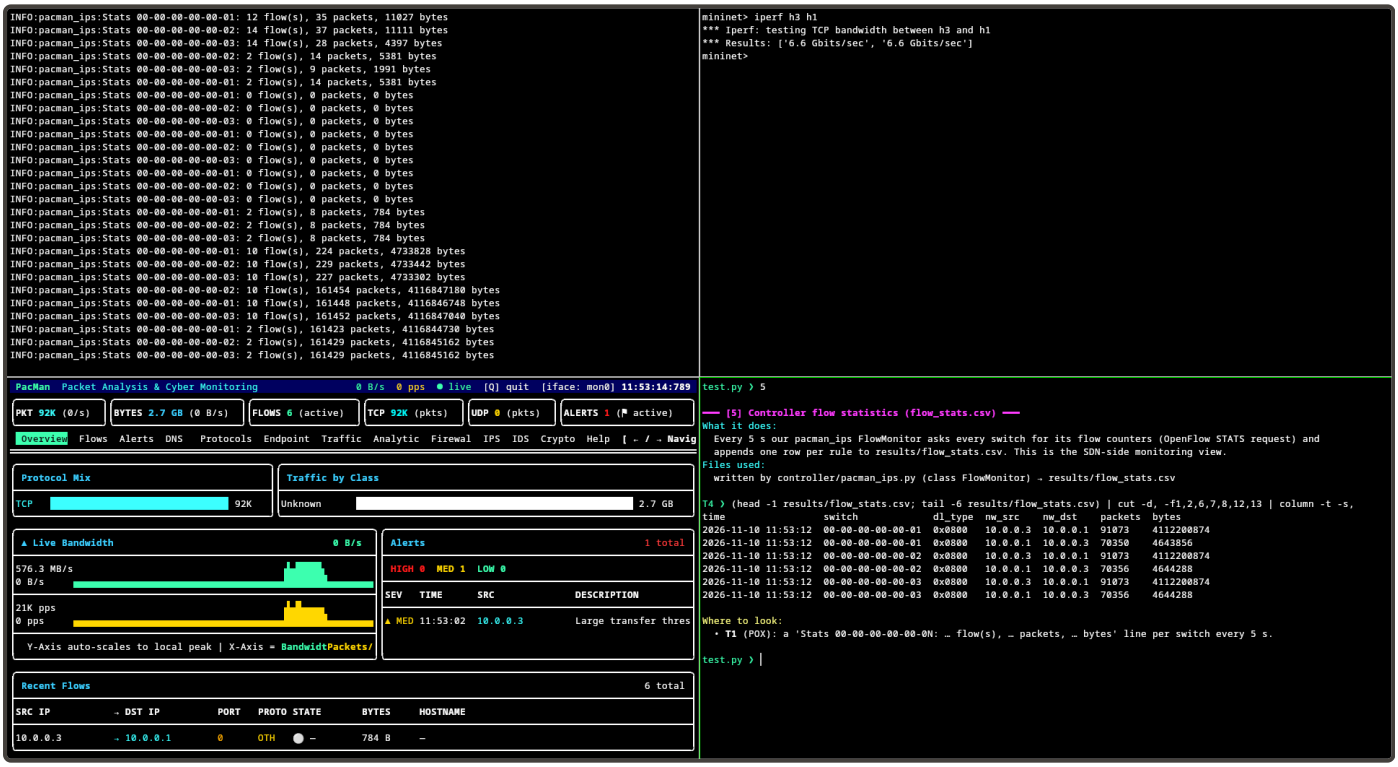


Figure 11. T2: iperf · T3: MEDIUM alert, no block · T4: flow statistics.

Step 13 · Normal traffic: no alert, no block

T4 · test.py ❯

```
6
```

DOES For 10 s, h3 requests h1's page and h4 requests h2's page once per second, then counts the lines in the blocklist.

FILES `scripts/on_host.sh` ; PacMan uses `config/pacman-sdn.conf` ; `results/blocklist.txt` stays empty

LOOK T4: 20 × HTTP 200, `blocklist lines: 0` . T3: packets grow, Alerts still **HIGH 0**. T1: nothing new.

SAY "Normal users browsing do not trigger anything. Even h4, our future attacker, is treated as normal while it behaves normally."

Q: Why doesn't the busy web server look like a scanner?
PacMan only counts connections a host *starts*. The server only *answers*, so its count stays 0.



Figure 12. Normal browsing: no HIGH alert, empty blocklist.

Part D · Attack and automatic blocking

Step 14 · Start a running ping from the attacker

T2 · mininet>

```
h4 ping -i 0.5 10.0.0.2
```

DOES h4 pings the file server h2 twice a second and keeps going. Leave it running.

SAY "This ping is h4's existing connection. Watch it: when we block h4, it should stop immediately."

Step 15 · ATTACK: h4 port-scans the web server

T4 · test.py ❯

```
7
```

DOES Runs `scripts/on_host.sh h4 python3 scripts/attacks/portscan.py 10.0.0.1 --ports 1-100 --timeout 0.2`: h4 tries to connect to ports 1–100 of h1.

INSIDE **Scanner:** for each port it opens a socket and calls `connect_ex()` (0 = open); it only accepts 10.0.0.x targets.
PacMan: each probe is a SYN that h4 *starts*; more than 15 different ports on h1 within 10 s → **PORT_SCAN (HIGH)** → appends `10.0.0.4 <time> Possible port scan detected` to blocklist.txt.
POX (Blocker): reads the new line within 1 s, checks it, and sends 2 messages to every switch: **DELETE** h4's existing flows, then **ADD** `ip, nw_src=10.0.0.4, priority 65000, no actions (= drop), hard_timeout 120`.

FILES `scripts/on_host.sh` → `scripts/attacks/portscan.py`; PacMan (thresholds in `config/pacman-sdn.conf`) → `results/pacman.log`, `results/blocklist.txt`; `controller/pacman_ips.py` (Blocker) → OpenFlow rules + `results/block_events.csv`

LOOK **T3:** Alerts box: **HIGH 10.0.0.4 Possible port scan detected** (+ MEDIUM high connection count). **T1: WARNING … BLOCKED 10.0.0.4 on 3 switch(es) for 119 s. T2:** the h4 ping **freezes. T4:** "scanned 100 ports … open: none" (the probes after the block time out, so the scan takes ~11–16 s).

SAY "h4 starts scanning. In about one second PacMan raises a HIGH alert, the controller blocks h4 on all three switches, and h4's ping stops. Nobody touched anything: it is automatic."

Q: How fast is it?
Median 1.28 s from scan start to rules on all switches (15 runs).
Q: Why did the ping stop at once?
Because we first DELETE h4's existing flow rules; otherwise the old exact-match rule would keep forwarding it.

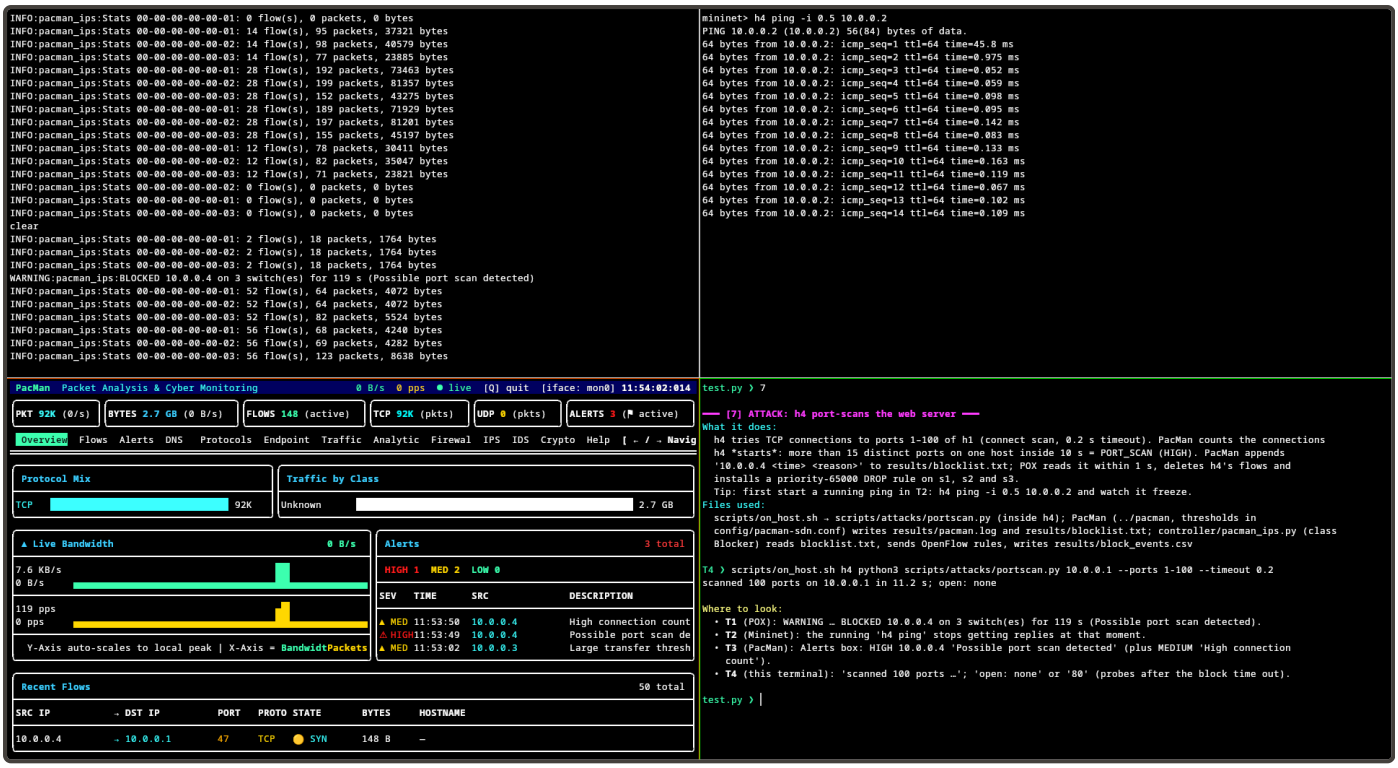


Figure 13. T1: BLOCKED · T2: h4's ping frozen at seq 14 · T3: HIGH alert · T4: the attack test.

Step 16 · Proof: the request and the DROP rules

T4 · test.py ❯

```
8
```

Then click T3 and press → 8 times to open the **Firewall** tab.

DOES `cat results/blocklist.txt`, then `ovs-ofctl dump-flows` for `ip,nw_src=10.0.0.4` on s1, s2, s3.

FILES `results/blocklist.txt` (written by PacMan); rules from `controller/pacman_ips.py`

LOOK T4: `10.0.0.4 1794… Possible port scan detected`; on every switch `hard_timeout=120, priority=65000, ip, nw_src=10.0.0.4 actions=drop`. T3 Firewall tab: **SDN_DROP 10.0.0.4**, enforcement "SDN CONTROLLER".

SAY "This one line is how PacMan talks to the controller. The controller turned it into a drop rule on every switch, for 2 minutes."

Q: Why a text file?

Simplest link between a C++ program and a Python 2.7 controller; it survives restarts and works as a log.

Q: Why 'ip' in the rule?

`dl_type=0x0800` (IPv4) is needed so the source-IP match works; without it the rule would drop everything.

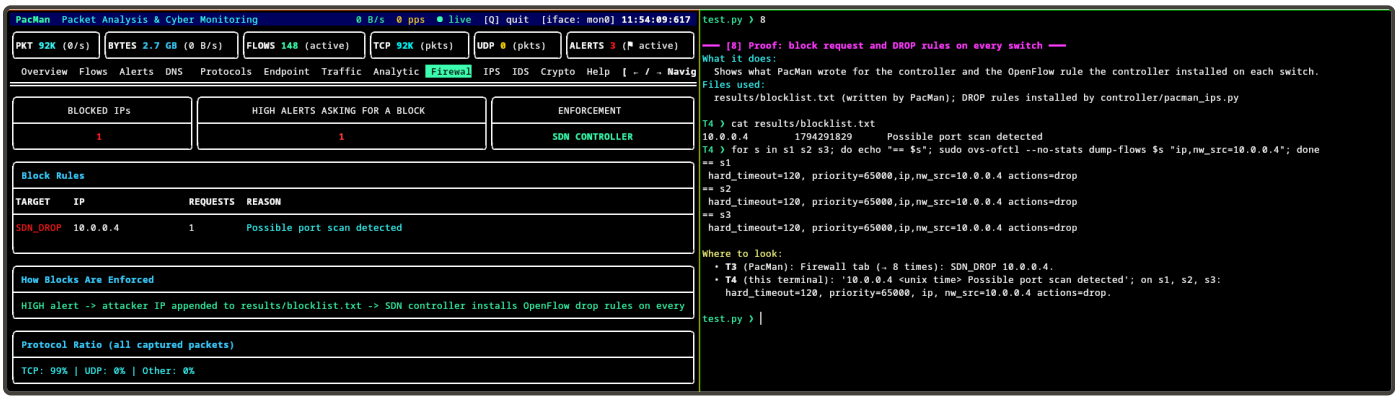


Figure 14. T3: Firewall tab · T4: blocklist line and DROP rules on s1, s2, s3.

Step 17 · Attacker cut off, user unaffected

T4 · test.py ❯

```
9
```

DOES h4 pings h2; h3 pings h1; h3 fetches h1's web page.

LOOK T4: h4 **100% packet loss** (even to the other server); h3 **0% loss** and **HTTP 200**.

SAY "Only the attacker is cut off. The normal user keeps working; there is no collateral damage."

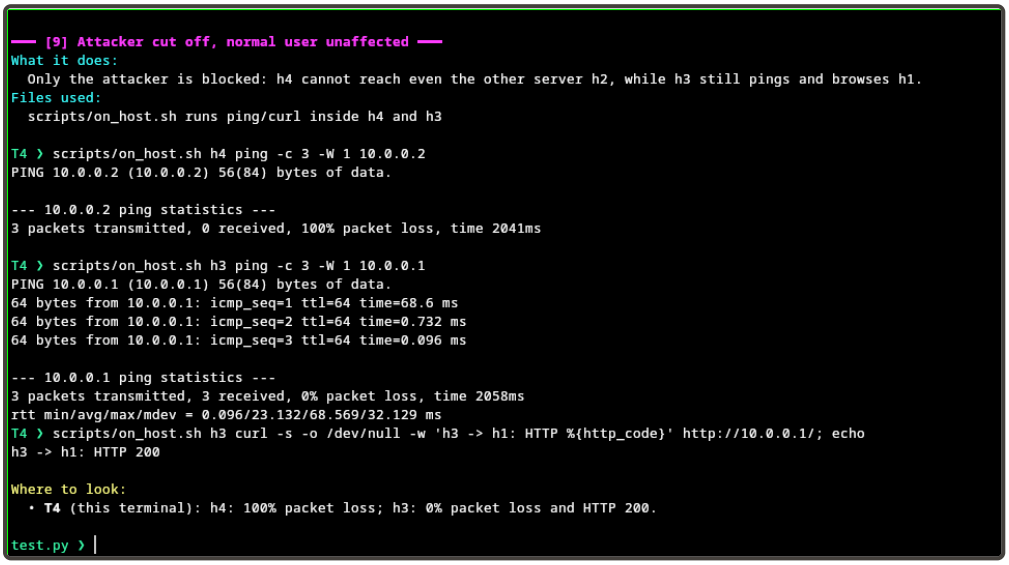


Figure 15. Attacker blocked, user fine.

Step 18 · Live DROP counter at the attacker's own switch

T4 · test.py ❯

```
10
```

DOES Prints the packet counter of the DROP rule on s3 once a second for 6 s.

LOOK T4: `n_packets` grows by ~2 per second, which is h4's frozen ping (T2) being dropped.

SAY "The attacker's packets are thrown away at its own edge switch, before they ever reach the core."

```
test.py ❯ 10

── [10] Live DROP counter on the attacker's edge switch s3 ──
What it does:
  Prints s3's drop-rule packet counter every second for 6 s. If h4 is still pinging (T2), the number climbs:
  the packets die at h4's own edge switch and never reach the core.
Files used:
  reads the DROP rule's counter from switch s3 (rule from controller/pacman_ips.py)

T4 ❯ for i in 1 2 3 4 5 6; do echo "$(date +%T)  s3 drop rule: $(sudo ovs-ofctl dump-flows s3 ip,nw_src=10.0.0.4 | grep
 actions=drop | grep -o 'n_packets=[0-9]*' || echo 'no DROP rule (not blocked)')"; sleep 1; done
11:54:22  s3 drop rule: n_packets=116
11:54:23  s3 drop rule: n_packets=119
11:54:24  s3 drop rule: n_packets=121
11:54:25  s3 drop rule: n_packets=123
11:54:26  s3 drop rule: n_packets=125
11:54:27  s3 drop rule: n_packets=127

Where to look:
  • T4 (this terminal): n_packets grows by ~2 per second while 'h4 ping -i 0.5' runs in T2.

test.py ❯
```

Figure 16. Drop counter rising.

Step 19 · The mirror sees no IP traffic from h4

T4 · test.py ❯

```
11
```

DOES `sudo timeout 5 tcpdump -nn -i mon0 ip and src host 10.0.0.4`: listens 5 s for IP packets from h4 on the core's mirror.

LOOK T4: **0 packets captured**.

SAY "Even though h4 is still pinging, none of its packets reach the core."

Q: Does ARP from h4 still pass?

Yes: the rule matches IPv4 only. ARP is harmless and lets h4 work normally again once the block ends.

```
test.py ❯ 11

── [11] Mirror sees no IP traffic from the attacker ──
What it does:
  Listens on mon0 for 5 s for IP packets from 10.0.0.4. They are dropped at s3 before the core, so nothing
  arrives. (ARP still passes: the rule matches IPv4 only, dl_type=0x0800.)
Files used:
  tcpdump on mon0 (mirror from topology/campus_topo.py)

T4 ❯ sudo timeout 5 tcpdump -nn -i mon0 ip and src host 10.0.0.4
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on mon0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

0 packets captured
0 packets received by filter
0 packets dropped by kernel

Where to look:
  • T3 (PacMan): packet rate from 10.0.0.4 stays at zero.
  • T4 (this terminal): '0 packets captured'.

test.py ❯
```

Figure 17. 0 packets from the attacker on mon0.

Step 20 · The controller's decision log

T4 · test.py ❯

```
12
```

DOES `column -t -s, results/block_events.csv`

FILES `results/block_events.csv` (written by `controller/pacman_ips.py`)

LOOK T4: `BLOCK 10.0.0.4 Possible port scan detected 3 119` (3 switches, 119 s).

SAY "Every decision is logged: block, refresh, expire, and also requests it refused, such as a protected IP or an invalid line."

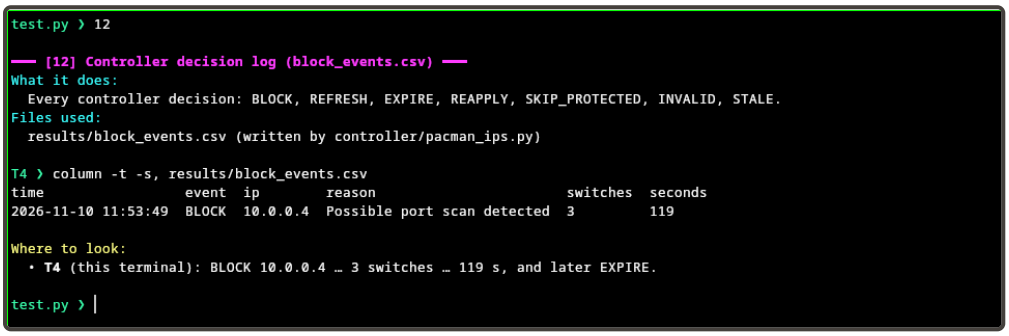


Figure 18. Block event.

Step 21 · The block expires by itself (waits up to 2 min)

T4 · test.py ❯

```
13
```

DOES Waits until `EXPIRE` appears in block_events.csv, then shows the events, checks that every switch has "no DROP rule", and pings from h4.

INSIDE The switches remove the rule themselves when its `hard_timeout` (120 s) runs out; the controller's 1 s timer logs EXPIRE.

LOOK T1: **"Block on 10.0.0.4 expired; traffic from it is allowed again"**. T2: **h4's ping starts again by itself**. T4: EXPIRE row, "no DROP rule" ×3, h4 0% loss.

SAY "The block is a 2-minute quarantine, not a permanent ban. If h4 scans again, it is blocked again."

Q: Why not block forever?
A wrong block heals itself, and the table does not fill up. A real attacker who keeps scanning is re-blocked within about a second.

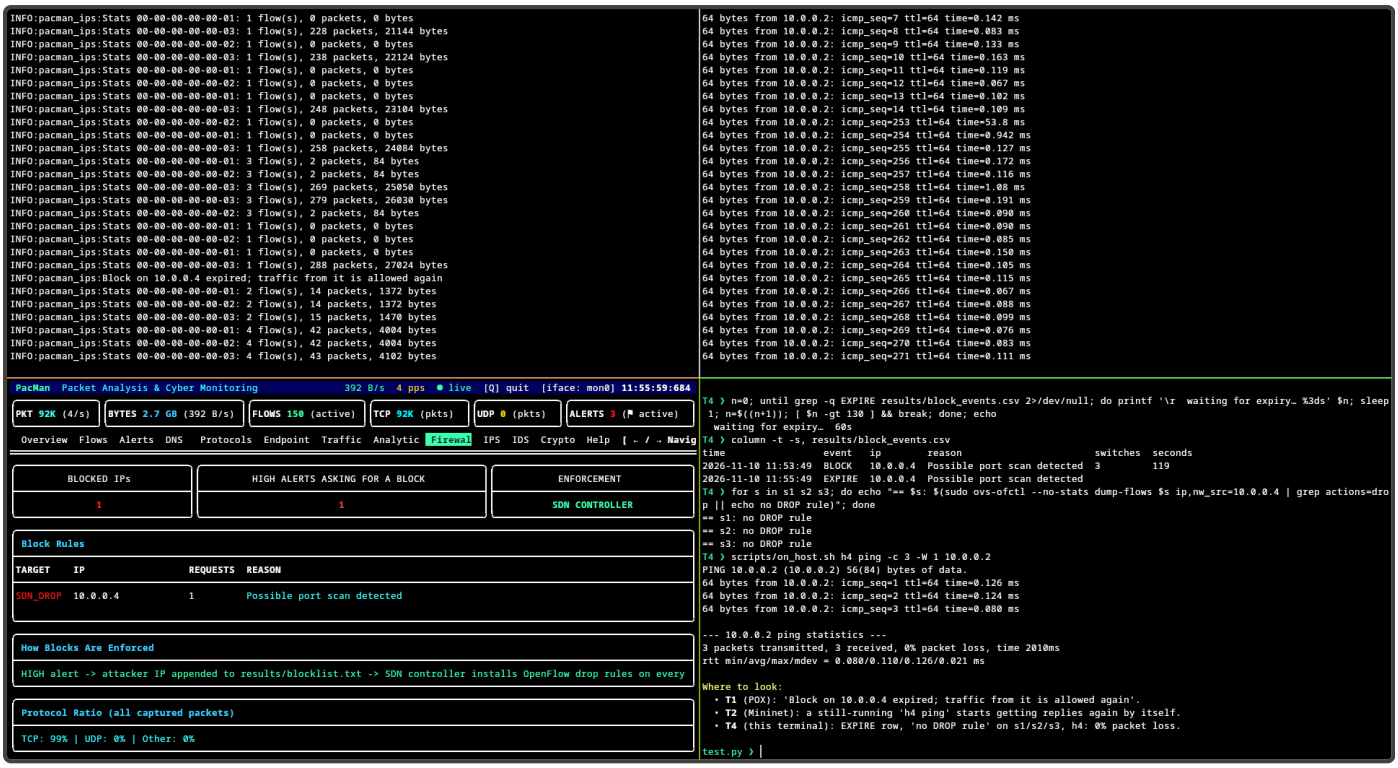


Figure 19. T1: expired · T2: ping resumes · T4: EXPIRE, no DROP rules, h4 reachable.

Step 22 · Stop the attacker's ping and read the summary

T2

```
Ctrl+C
```

LOOK T2: e.g. `272 packets transmitted, 34 received, 87.5% packet loss`. The ~238 lost pings ≈ 119 s × 2 per second, which is exactly the block time.

SAY "The numbers match: the attacker lost exactly two minutes of traffic."

```
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.102 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.109 ms
64 bytes from 10.0.0.2: icmp_seq=253 ttl=64 time=53.8 ms
64 bytes from 10.0.0.2: icmp_seq=254 ttl=64 time=0.942 ms
64 bytes from 10.0.0.2: icmp_seq=255 ttl=64 time=0.127 ms
64 bytes from 10.0.0.2: icmp_seq=256 ttl=64 time=0.172 ms
64 bytes from 10.0.0.2: icmp_seq=257 ttl=64 time=0.116 ms
64 bytes from 10.0.0.2: icmp_seq=258 ttl=64 time=1.08 ms
64 bytes from 10.0.0.2: icmp_seq=259 ttl=64 time=0.191 ms
64 bytes from 10.0.0.2: icmp_seq=260 ttl=64 time=0.090 ms
64 bytes from 10.0.0.2: icmp_seq=261 ttl=64 time=0.090 ms
64 bytes from 10.0.0.2: icmp_seq=262 ttl=64 time=0.085 ms
64 bytes from 10.0.0.2: icmp_seq=263 ttl=64 time=0.150 ms
64 bytes from 10.0.0.2: icmp_seq=264 ttl=64 time=0.105 ms
64 bytes from 10.0.0.2: icmp_seq=265 ttl=64 time=0.115 ms
64 bytes from 10.0.0.2: icmp_seq=266 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=267 ttl=64 time=0.088 ms
64 bytes from 10.0.0.2: icmp_seq=268 ttl=64 time=0.099 ms
64 bytes from 10.0.0.2: icmp_seq=269 ttl=64 time=0.076 ms
64 bytes from 10.0.0.2: icmp_seq=270 ttl=64 time=0.083 ms
64 bytes from 10.0.0.2: icmp_seq=271 ttl=64 time=0.111 ms
64 bytes from 10.0.0.2: icmp_seq=272 ttl=64 time=0.058 ms
^C
--- 10.0.0.2 ping statistics ---
272 packets transmitted, 34 received, 87.5% packet loss, time 136596ms
rtt min/avg/max/mdev = 0.052/3.107/53.757/11.715 ms
mininet>
```

Figure 20. Ping summary.

Part E · Failure, live cheat-sheet, stop

Step 23 · Cut a cable, then fix it

T2 · mininet>

```
link s1 s3 down
```

T2 · mininet>

```
h3 ping -c 3 -W 1 10.0.0.1
```

T2 · mininet>

```
link s1 s3 up
```

T2 · mininet>

```
h3 ping -c 3 -W 1 10.0.0.1
```

DOES Disconnects the core↔user-switch link, tests, reconnects, tests again.

LOOK T2: first ping 100% loss, second 0% loss.

SAY "If a cable breaks, the user zone is cut off; when it is back, traffic recovers by itself."

Q: Why no backup path?
Our tree has one path per zone. A real campus would add redundant links (with loop protection).

Step 24 · All live Mininet commands (cheat-sheet)

T4 · test.py ❯

```
live
```

DOES Prints every Mininet command for T2, grouped (look at the network, hosts as machines, connectivity, web servers, bandwidth, the attacker's running ping, failures, switch commands from Mininet), each with what it does. Copy any line into T2.

```
mininet> link s1 s3 down
mininet> h3 ping -c 3 -W 1 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.

--- 10.0.0.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2086ms

mininet> link s1 s3 up
mininet> h3 ping -c 3 -W 1 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=61.4 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.737 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.115 ms

--- 10.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2070ms
rtt min/avg/max/mdev = 0.115/20.751/61.401/28.745 ms
mininet>
```

```
       TCP bandwidth test through the core (~22 Gbit/s on this laptop). PacMan shows a MEDIUM 'Large
       transfer' alert for h3 and does NOT block it: only HIGH alerts block

The attacker's running connection (start BEFORE test 'attack')
  16  h4 ping -i 0.5 10.0.0.2
       h4 pings h2 twice a second and keeps going. When the block lands the replies stop; when it expires
       they come back. Stop with Ctrl+C: the summary shows the lost packets

Failures
  17  link s1 s3 down
       cut the cable between the core and the user switch
  18  h3 ping -c 3 -W 1 10.0.0.1
       100% packet loss: the user zone is cut off
  19  link s1 s3 up
       plug it back in
  20  h3 ping -c 3 -W 1 10.0.0.1
       0% loss again: the network recovers by itself

Switch and controller from inside Mininet
  21  sh ovs-ofctl dump-flows s3
       s3's flow table, run from the Mininet prompt (sh = run on the host)
  22  sh ovs-vsctl show
       all bridges, ports, the mirror and the controller connection

Finish
  23  exit
       stop Mininet (then run 'sudo mn -c' in T4 if anything is left over)

test.py ❯
```

Figure 21. T2: link down/up · T4: the live cheat-sheet.

Step 25 · Stop everything

Terminal	Action
T3 (PacMan)	press `q`
T2 (mininet>)	`exit`
T1 (POX)	press `Ctrl+C`
T4	`q` to leave test.py, then `sudo mn -c`

OPTIONAL Unblock early: `test.py ❯ 14`. Full automatic test suites (≈ 8 min, only with everything stopped): `scripts/run_all_tests.sh` → 21 + 25 + 8 = 54 checks.

Part F · What is inside each script

scripts/check_env.sh: is everything installed?

```
for cmd in mn ovs-vsctl ovs-ofctl podman tcpdump python3; do command -v "$cmd" ... ok / bad
sudo -n true                                  # passwordless sudo?
sudo python3 -c "import mininet"              # Mininet Python API as root
systemctl is-active openvswitch               # switch service running
podman image exists python:2.7                # controller image
[ -f ../pox/pox.py ] ; [ -x ../pacman/build/pacman ]
ss -ltn | grep ':6633 '                       # is the OpenFlow port free?
```

scripts/run_pox.sh: start the controller

```
STATS_INTERVAL=${STATS_INTERVAL:-5}  BLOCK_SECONDS=${BLOCK_SECONDS:-120}  PROTECT=${PROTECT-10.0.0.1}
podman rm -f pacman-pox                        # remove an old one
podman run --rm --network host --security-opt label=disable \
  -v ../pox:/pox  -v controller:/project/controller:ro  -v results:/project/results \
  python:2.7  python ./pox.py log.level --INFO forwarding.l2_learning \
     pacman_ips --interval=5 --stats_file=… --blocklist=… --block_seconds=120 --protect=10.0.0.1 --
events_file=…
```

topology/campus_topo.py: build the network

```
HOSTS = {"h1": ("10.0.0.1","s2","web server"), "h2": ("10.0.0.2","s2","file server"),
         "h3": ("10.0.0.3","s3","normal user"), "h4": ("10.0.0.4","s3","attacker")}
build():     s1,s2,s3 = addSwitch(...); addLink(s1,s2); addLink(s1,s3); addHost(ip, mac) + addLink(host,
switch)
build_net(): Mininet(topo, OVSSwitch) + RemoteController(127.0.0.1:6633); disable IPv6 on hosts and switch
ports
add_mirror(): add internal port mon0 on s1, create mirror select-all → mon0, mon0 no-flood
main():      waitConnected → print hosts → CLI(net) → net.stop()
```

scripts/run_pacman.sh: start PacMan

```
[ -x ../pacman/build/pacman ] || error;   ip link show mon0 || "start the topology first"
mkdir -p results/pacman-reports; touch results/pacman.log results/blocklist.txt   # owned by you, not root
exec sudo ../pacman/build/pacman live config/pacman-sdn.conf --iface mon0
```

config/pacman-sdn.conf: PacMan settings

```
interface=mon0   detection_window_s=10   port_scan_threshold=15   syn_flood_threshold=200
high_connection_threshold=40   large_transfer_bytes=10485760
firewall_mode=sdn   blocklist_file=results/blocklist.txt   block_cooldown_s=30
```

scripts/on_host.sh: run a command inside a Mininet host

```
pid=$(pgrep -f "^bash --norc --noediting -is mininet:h4$")   # h4's shell, started by Mininet
exec sudo mnexec -a "$pid" python3 scripts/attacks/portscan.py ...   # enter h4's namespaces and run
```

scripts/attacks/portscan.py: the attacker

```
refuse any target outside 10.0.0.0/24
for port in 1..100:  s = socket(); s.settimeout(0.2); if s.connect_ex((target, port)) == 0:
open.append(port)
print("scanned 100 ports on 10.0.0.1 in X s; open: ...")
```

Inside PacMan (C++): how a scan is detected

```
capture thread:   pcap_open_live("mon0") → copy each packet into a queue of 10,000
processing:       parse Ethernet / IPv4 / TCP → flow table (5-tuple) → is it "initiated"?
initiated (TCP) = first packet of the flow AND SYN set AND ACK not set      (replies never count)
window 10 s:      ports[h4->h1] = {21, 22, 23, ...}   when size > 15  → alert PORT_SCAN, severity HIGH
HIGH + sdn mode:  append "10.0.0.4 TAB unix_time TAB reason" to results/blocklist.txt (once per 30 s per IP)
```

Inside controller/pacman_ips.py: FlowMonitor and Blocker

```
FlowMonitor  every 5 s:  send ofp_stats_request(flow) to every switch → write rows to results/flow_stats.csv
Blocker      every 1 s:  read only NEW complete lines of blocklist.txt (remembers offset; half lines wait)
             check:      valid IPv4? protected? older than block time? → INVALID / SKIP_PROTECTED / STALE
             block:      for every switch:  FLOW_MOD DELETE  match(dl_type=0x0800, nw_src=ip)
                                            FLOW_MOD ADD     same match, priority=65000, no actions,
hard_timeout=120
             log:        BLOCK / REFRESH / EXPIRE / REAPPLY → results/block_events.csv
             switch reconnects or POX restarts → still-valid blocks are installed again
```

test.py: the T4 console

```
TESTS = [{key, title, why, files, cmds, look}, ...]       15 tests (0-14) + LIVE cheat-sheet
menu → input number/name → print "What it does" + "Files used" → for each cmd: print "T4 ❯ cmd", run it in
bash
     → print "Where to look" for T1/T2/T3/T4      ('clear' clears, Enter = menu, q = quit)
python3 test.py 7     # run a single test directly;   python3 test.py live   # just the cheat-sheet
```

Quick cross-question sheet (whole demo)

Q: What exactly is SDN here?

Switches only forward; one controller (POX) decides for all of them. That's why one decision blocks the attacker everywhere.

Q: What does PacMan add that the controller can't see?

Every packet's details, such as TCP flags. So it knows who *started* a connection; the controller only sees counters.

Q: What if PacMan crashes?

The network keeps working (PacMan only watches a copy). Blocks already in place stay until they expire.

Q: What if POX crashes?

The switches keep their rules; new flows aren't forwarded (fail-secure). After a restart POX re-reads the blocklist and re-installs still-valid blocks.

Q: Can a slow scan escape?

Yes, below 1.5 ports per second (15 ports in 10 s). That's a known limit; future work is TRW-style scoring.

Q: Can the attacker get the server blocked by faking its IP?

No: 10.0.0.1 is on the protected list. For other hosts, the block is temporary (2 min).

Q: How did you test all this?

54 automatic checks in 3 suites (network, detection and blocking, load), all passing; each suite starts from a clean state.